

**IJSE**

Comprehensive Master Java Developer (CMJD)

Batch 110/111

# **Research Project Tracker**

for an Educational Institute

*Coursework Technical Project Report*

## **Assignment 1**

**Backend Development with Spring Boot and MySQL**

(Spring Boot + Spring Data JPA + Spring Security/JWT + MySQL)

## **Assignment 2**

**Front End Development with React**

(React + TypeScript + React Router + Axios + Context API + React Bootstrap)

Submitted by

**Gnei Shameena Bawa**

*[ St\_ID 25021111060 / NIC 957440380 V ]*

15 August 2026

## Abstract

This document presents the design, implementation, integration, and testing plan for the **Research Project Tracker for an Educational Institute**. The system combines a Spring Boot backend with a MySQL persistence layer and stateless JWT-based security, and a React/TypeScript frontend that communicates with the backend through Axios. The documented implementation supports user registration and login, role-based access control, project management, milestone management, document upload and management, and administrator-only user management. The technical requirements are grounded in the supplied CMJD backend and frontend coursework briefs and the current GitHub repository.



## Table of Contents

---

|                                                              |           |
|--------------------------------------------------------------|-----------|
| <b>Abstract .....</b>                                        | <b>2</b>  |
| <b>Table of Contents.....</b>                                | <b>4</b>  |
| <b>1. Introduction .....</b>                                 | <b>6</b>  |
| 1.1 Project Overview .....                                   | 6         |
| 1.2 Objective.....                                           | 6         |
| 1.3 Problem Being Solved.....                                | 6         |
| <b>2. Technology Stack.....</b>                              | <b>7</b>  |
| 2.1 Backend .....                                            | 7         |
| 2.2 Frontend .....                                           | 8         |
| 2.3 Development / Version Control.....                       | 8         |
| <b>3. System Architecture and Design Patterns .....</b>      | <b>9</b>  |
| 3.1 System Architecture .....                                | 9         |
| 3.2 System Design Patterns .....                             | 9         |
| 3.3 Backend Package Structure.....                           | 11        |
| 3.4 Request Flow .....                                       | 11        |
| <b>4. Data Model .....</b>                                   | <b>11</b> |
| 4.1 Data Dictionary and Relationships .....                  | 11        |
| 4.2 Entities .....                                           | 12        |
| 4.2.1 User.....                                              | 12        |
| 4.2.2 Project.....                                           | 12        |
| 4.2.3 Milestone .....                                        | 13        |
| 4.2.4 Document.....                                          | 13        |
| <b>5. Functional and Non – Functional Requirements .....</b> | <b>13</b> |
| 5.1 Functional Requirements .....                            | 13        |
| 5.2 Non-Functional Requirements.....                         | 14        |
| <b>6. Core Features and Implementation .....</b>             | <b>15</b> |
| 6.1 Authentication and Authorization .....                   | 15        |
| 6.2 Project Management .....                                 | 15        |
| 6.3 Milestone Management .....                               | 15        |
| 6.4 Document Management .....                                | 15        |
| 6.5 Admin Panel.....                                         | 15        |
| <b>6. Security Implementation.....</b>                       | <b>16</b> |
| 6.1 Security Design .....                                    | 16        |
| 6.2 Authentication Mechanism.....                            | 16        |
| 6.3 Role-Based Access Control .....                          | 16        |
| 6.4 Error Handling .....                                     | 17        |
| <b>7. Backend API Reference (REST API Design) .....</b>      | <b>18</b> |

|             |                                                                            |           |
|-------------|----------------------------------------------------------------------------|-----------|
| <b>7.1</b>  | <b>Spring Boot Project Setup and Configuration .....</b>                   | <b>18</b> |
| <b>7.2</b>  | <b>Authentication and JWT Flow .....</b>                                   | <b>18</b> |
| <b>7.3</b>  | <b>Project Management .....</b>                                            | <b>18</b> |
| <b>7.4</b>  | <b>Milestone Management .....</b>                                          | <b>19</b> |
| <b>7.5</b>  | <b>Document Management .....</b>                                           | <b>19</b> |
| <b>7.6</b>  | <b>User Administration .....</b>                                           | <b>20</b> |
| <b>7.7</b>  | <b>Exception Handling and Supporting Components .....</b>                  | <b>20</b> |
| <b>8.</b>   | <b>Frontend Application Structure .....</b>                                | <b>21</b> |
| <b>8.1</b>  | <b>Routing and Navigation .....</b>                                        | <b>21</b> |
| <b>8.2</b>  | <b>Key Design Decisions .....</b>                                          | <b>21</b> |
| <b>8.3</b>  | <b>Frontend Implementation.....</b>                                        | <b>22</b> |
| 8.3.1       | React Project Setup and Route Composition .....                            | 22        |
| 8.3.2       | Authentication Context and Token Lifecycle .....                           | 22        |
| 8.3.3       | Login and Registration .....                                               | 22        |
| 8.3.4       | Project Pages and Dashboard.....                                           | 22        |
| 8.3.5       | Project Details, Status, Milestones, and Documents.....                    | 23        |
| 8.3.6       | Role-Based UI and Administration.....                                      | 23        |
| 8.3.7       | Responsive Design, Loading, and Error Feedback .....                       | 23        |
| <b>9.</b>   | <b>Application Screenshots .....</b>                                       | <b>24</b> |
| <b>9.1</b>  | <b>Sign In .....</b>                                                       | <b>24</b> |
| <b>9.2</b>  | <b>Sign Up .....</b>                                                       | <b>25</b> |
| <b>9.3</b>  | <b>Dashboard .....</b>                                                     | <b>26</b> |
| <b>9.4</b>  | <b>Projects List.....</b>                                                  | <b>28</b> |
| <b>9.5</b>  | <b>Project Detail.....</b>                                                 | <b>30</b> |
| <b>9.6</b>  | <b>Admin Panel.....</b>                                                    | <b>34</b> |
| <b>10.</b>  | <b>Setup and Installation .....</b>                                        | <b>36</b> |
| <b>10.1</b> | <b>Prerequisites.....</b>                                                  | <b>36</b> |
| <b>10.2</b> | <b>Backend .....</b>                                                       | <b>36</b> |
| <b>10.3</b> | <b>Frontend .....</b>                                                      | <b>36</b> |
| <b>11.</b>  | <b>System Integration, Deployment Environment and Version Control.....</b> | <b>37</b> |
| 11.1        | End-to-End Integration Flow .....                                          | 37        |
| 11.2        | Version Control and Run Environment .....                                  | 37        |
| <b>12.</b>  | <b>Conclusion .....</b>                                                    | <b>38</b> |
| <b>13.</b>  | <b>Recommendations.....</b>                                                | <b>39</b> |

# 1. Introduction

---

## 1.1 Project Overview

The Research Project Tracker is a full-stack web application built for an educational institute to manage academic research projects. It allows Principal Investigators, research members, administrators, and viewers to create, monitor, and collaborate on research projects, track milestones against deadlines, and share supporting documents, all behind a secure, role-based authentication system.

## 1.2 Objective

The objective of this coursework was to design and implement a multi-entity CRUD system with proper data relationships, secure the API using Spring Security and JWT, apply role-based access control in a research collaboration context, and build a responsive React front end that consumes that API covering the full stack from database schema to user interface.

- Support secure sign-up and sign-in using stateless JWT authentication and BCrypt password hashing.
- Provide role-based capabilities for ADMIN, PI, MEMBER, and VIEWER users.
- Enable creation, viewing, updating, status changes, and deletion of research projects according to authorization rules.
- Enable project milestones to be created, updated, completed, and deleted by authorized users.
- Enable project documents to be uploaded, listed, opened, and deleted under the defined role rules.
- Provide an administrator panel for user listing and deletion.
- Expose a responsive React interface that uses protected routes, role-aware controls, loading feedback, and friendly error handling.
- Persist core domain data in MySQL using Spring Data JPA and store uploaded files using the backend file-storage component.
- Maintain source code and setup documentation in GitHub as required by the coursework.

## 1.3 Problem Being Solved

Educational institutes running multiple concurrent research projects need a central system to track project status, assign ownership to Principal Investigators, record milestones and deadlines, and manage supporting documentation, while ensuring that only appropriately authorized users can create, modify, or delete sensitive data. This system addresses that need directly.

## 2. Technology Stack

### 2.1 Backend

| <b>Layer / Component</b> | <b>Technology</b>                                              | <b>Role in System</b>                                                  |
|--------------------------|----------------------------------------------------------------|------------------------------------------------------------------------|
| <b>Backend Runtime</b>   | Java 17                                                        | Primary backend language/runtime.                                      |
| <b>Backend Framework</b> | Spring Boot 3.3.4                                              | REST application, dependency injection, configuration.                 |
| <b>Web Layer</b>         | Spring Web                                                     | REST controllers and HTTP request/response handling.                   |
| <b>Persistence</b>       | Spring Data JPA + Hibernate                                    | Entity mapping, repositories, CRUD persistence.                        |
| <b>Database</b>          | MySQL 8 (accessed via Spring Data JPA / Hibernate)             | Relational persistence for users, projects, milestones, and documents. |
| <b>Security</b>          | Spring Security 6 + stateless JWT authentication (jjwt 0.11.5) | Stateless authentication and authorization.                            |
| <b>Password Security</b> | BCrypt                                                         | Password hashing before persistence.                                   |
| <b>Build Tool</b>        | Maven                                                          | Dependency and build management.                                       |
| <b>Other</b>             | Lombok, Bean Validation                                        | Jakarta Validation                                                     |

## 2.2 Frontend

| <b>Layer / Component</b>    | <b>Technology</b>        | <b>Role in System</b>                                |
|-----------------------------|--------------------------|------------------------------------------------------|
| <b>Framework</b>            | React 18 with TypeScript | Typed SPA user interface(Create React App Template). |
| <b>Routing</b>              | React Router v6          | Client-side routes and protected navigation.         |
| <b>HTTP Client</b>          | Axios                    | Backend communication and JWT interceptors.          |
| <b>State management</b>     | React Context API        | Global authentication and session state.             |
| <b>UI Library (Styling)</b> | React Bootstrap          | Responsive UI components and forms.                  |

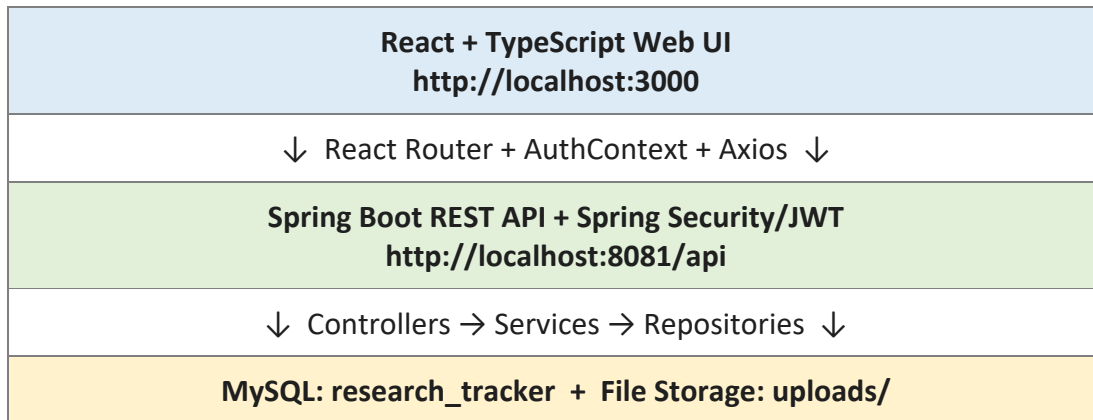
## 2.3 Development / Version Control

| <b>Layer / Component</b> | <b>Technology</b> | <b>Role in System</b>                            |
|--------------------------|-------------------|--------------------------------------------------|
| Version Control          | GitHub / Git      | Repository management and coursework submission. |

### 3. System Architecture and Design Patterns

#### 3.1 System Architecture

The system follows a client-server layered architecture. React handles the presentation and client-side navigation, Axios acts as the HTTP integration layer, Spring Boot exposes authenticated REST endpoints, service classes implement business rules, Spring Data repositories manage persistence, and MySQL stores domain data. Uploaded document files are stored by the backend file-storage service and exposed through the configured uploads path.



*Architecture overview of the Research Project Tracker*

#### 3.2 System Design Patterns

| Pattern               | How It Is Applied                                                                                                                                                                                 |
|-----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Layered Architecture  | Backend separates controller, service, repository/entity, configuration, and common exception concerns. This reduces coupling and makes authorization/business rules easier to test and maintain. |
| Repository Pattern    | Spring Data JPA repositories encapsulate persistence operations for User, Project, Milestone, and Document entities.                                                                              |
| Service Layer Pattern | ProjectService, MilestoneService, DocumentService, UserService, and AuthenticationService centralize business logic rather than embedding it in controllers.                                      |

| Pattern                    | How It Is Applied                                                                                                                                 |
|----------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| Dependency Injection       | Spring-managed components are injected through constructor-based dependency injection, reducing manual object creation and improving testability. |
| DTO Pattern                | Request/response DTO packages decouple API payloads from persistence entities and support validation.                                             |
| Provider / Context Pattern | AuthContext exposes user, token, authenticated state, login, register, and logout to the React component tree.                                    |
| Route Guard Pattern        | PrivateRoute and RoleRoute prevent unauthenticated or unauthorized navigation at the client level.                                                |
| Interceptor Pattern        | Axios interceptors attach JWTs to requests and centralize the 401 redirect/cleanup behavior.                                                      |
| Role-Based Access Control  | ADMIN, PI, MEMBER, and VIEWER permissions are enforced in backend methods and reflected in frontend controls.                                     |

### 3.3 Backend Package Structure

The backend follows a feature-based package structure rather than a purely technical layered one, keeping each domain's entity, repository, service, controller, and DTOs together.

- `auth/` → signup and login (AuthenticationController, AuthenticationService)
- `project/` → Project entity, repository, service, controller, DTOs
- `milestone/` → Milestone entity, repository, service, controller, DTOs
- `document/` → Document entity, file storage service, controller, DTOs
- `user/` → User entity (implements Spring Security's UserDetails), admin user management
- `config/` → Spring Security configuration, JWT filter and service, CORS, startup data seeder
- `common/exception/` → centralised REST exception handling

### 3.4 Request Flow

Every request to a protected endpoint passes through a `JwtAuthenticationFilter`, which validates the JWT from the Authorization header and populates Spring Security's context before the request reaches the controller. Controllers delegate all business logic to a service layer, which enforces both role checks (via `@PreAuthorize`) and ownership checks (for example, confirming a Principal Investigator owns the project they are trying to edit) before touching the repository layer.

## 4. Data Model

### 4.1 Data Dictionary and Relationships

| Entity    | Key Fields                                                                     | Purpose / Relationship                                                                      |
|-----------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------|
| User      | id, username, password, fullName, role, createdAt                              | Represents ADMIN, PI, MEMBER, or VIEWER. Referenced by projects, milestones, and documents. |
| Project   | id, title, summary, status, pi, tags, startDate, endDate, createdAt, updatedAt | Academic research project. PI is linked to a User.                                          |
| Milestone | id, project, title, description, dueDate, isCompleted, createdBy               | Project deliverable/stage linked to a Project and creating User.                            |

| Entity   | Key Fields                                                         | Purpose / Relationship                                             |
|----------|--------------------------------------------------------------------|--------------------------------------------------------------------|
| Document | id, project, title, description, urlOrPath, uploadedBy, uploadedAt | Metadata for a project file plus the server path/URL and uploader. |

| Relationship        | Cardinality | Description                                                          |
|---------------------|-------------|----------------------------------------------------------------------|
| User → Project      | 1 : many    | A PI user can own multiple projects; each project references one PI. |
| Project → Milestone | 1 : many    | A project contains zero or more milestones.                          |
| User → Milestone    | 1 : many    | Each milestone records the user who created it.                      |
| Project → Document  | 1 : many    | A project contains zero or more document records.                    |
| User → Document     | 1 : many    | Each document records the user who uploaded it.                      |

## 4.2 Entities

### 4.2.1 User

| Field     | Type          | Notes                                       |
|-----------|---------------|---------------------------------------------|
| id        | String        | UUID, primary key                           |
| username  | String        | Unique login identifier                     |
| password  | String        | BCrypt hash, never exposed in API responses |
| fullName  | String        | Display name                                |
| role      | Enum          | ADMIN, PI, MEMBER, or VIEWER                |
| createdAt | LocalDateTime | Set automatically on creation               |

### 4.2.2 Project

| Field               | Type      | Notes                                          |
|---------------------|-----------|------------------------------------------------|
| id                  | String    | UUID, primary key                              |
| title / summary     | String    | Core descriptive fields                        |
| status              | Enum      | PLANNING, ACTIVE, ON_HOLD, COMPLETED, ARCHIVED |
| pi                  | User      | Linked Principal Investigator (many-to-one)    |
| tags                | String    | Comma-separated                                |
| startDate / endDate | LocalDate | Project timeline                               |

| Field                 | Type          | Notes                      |
|-----------------------|---------------|----------------------------|
| createdAt / updatedAt | LocalDateTime | Audit fields, auto-managed |

### 4.2.3 Milestone

| Field               | Type      | Notes                        |
|---------------------|-----------|------------------------------|
| id                  | String    | UUID, primary key            |
| project             | Project   | Parent project (many-to-one) |
| title / description | String    | Milestone details            |
| dueDate             | LocalDate | Deadline                     |
| isCompleted         | Boolean   | Completion flag              |
| createdBy           | User      | Who added the milestone      |

### 4.2.4 Document

| Field               | Type          | Notes                          |
|---------------------|---------------|--------------------------------|
| id                  | String        | UUID, primary key              |
| project             | Project       | Parent project (many-to-one)   |
| title / description | String        | File label and notes           |
| urlOrPath           | String        | Server path to the stored file |
| uploadedBy          | User          | Who uploaded the file          |
| uploadedAt          | LocalDateTime | Set automatically              |

## 5. Functional and Non – Functional Requirements

### 5.1 Functional Requirements

| ID   | Requirement     | Description                                                           |
|------|-----------------|-----------------------------------------------------------------------|
| FR01 | Registration    | Allow a new user to create an account; default role is MEMBER.        |
| FR02 | Login           | Authenticate username/password and return a JWT.                      |
| FR03 | Project List    | List projects for authenticated users.                                |
| FR04 | Project Details | Display project metadata, PI, dates, tags, milestones, and documents. |
| FR05 | Create Project  | PI and ADMIN can create a project.                                    |

| ID   | Requirement          | Description                                                                                                     |
|------|----------------------|-----------------------------------------------------------------------------------------------------------------|
| FR06 | Update Project       | Owning PI and ADMIN can update project details.                                                                 |
| FR07 | Update Status        | Owning PI and ADMIN can change project status.                                                                  |
| FR08 | Delete Project       | ADMIN can delete a project.                                                                                     |
| FR09 | Milestone Management | Authorized MEMBER/PI/ADMIN users can add milestones; creator/owning PI/ADMIN can modify based on backend rules. |
| FR10 | Document Management  | Authorized MEMBER/PI/ADMIN users can upload documents; owning PI/ADMIN can delete documents.                    |
| FR11 | User Administration  | ADMIN can list users and delete users.                                                                          |
| FR12 | Logout               | Clear the stored JWT and return the user to an unauthenticated state.                                           |
| FR13 | Protected Navigation | Unauthenticated users are redirected to login; ADMIN route is restricted.                                       |
| FR14 | Error Feedback       | Show validation, authorization, and API errors in a user-friendly form.                                         |

## 5.2 Non-Functional Requirements

| ID    | Area            | Requirement                                                                                                                 |
|-------|-----------------|-----------------------------------------------------------------------------------------------------------------------------|
| NFR01 | Security        | Stateless JWT authentication; BCrypt hashing; protected routes; role-based authorization.                                   |
| NFR02 | Usability       | Consistent layout, form validation, clear navigation, status badges, loading indicators, and understandable error messages. |
| NFR03 | Responsiveness  | Usable on desktop and mobile browser sizes using React Bootstrap/responsive CSS.                                            |
| NFR04 | Maintainability | Layered packages on backend; reusable React components, typed models, and Context API on frontend.                          |
| NFR05 | Persistence     | JPA/Hibernate persistence to MySQL with UUID identifiers and automatic timestamps where applicable.                         |
| NFR06 | Reliability     | Graceful API failures and centralized exception responses rather than raw stack traces.                                     |
| NFR07 | Performance     | Normal local CRUD actions should respond interactively; document uploads are capped at 10 MB in current configuration.      |
| NFR08 | Compatibility   | Java 17+, MySQL 8, Node.js 18+, modern browsers; CORS allows the local React origin.                                        |
| NFR09 | Version Control | Source, README/run instructions, and meaningful Git history are maintained in GitHub.                                       |

## 6. Core Features and Implementation

---

### 6.1 Authentication and Authorization

New users self-register via `/api/auth/signup` and are assigned the MEMBER role by default. Login issues a signed JWT containing the user's id, username, full name, and role, which the frontend decodes and stores to drive both UI state and route access. All authentication is stateless, no server-side session is kept.

### 6.2 Project Management

Any authenticated user can browse all projects. Only PIs and Admins can create a project; PIs automatically become the project's owner (an Admin may assign a different PI). Updating a project or changing its status is restricted to that project's own PI or an Admin. Deleting a project is restricted to Admins only.

### 6.3 Milestone Management

Milestones belong to a project and can be created by Members, PIs, or Admins. A milestone can be edited or deleted by whoever created it, the owning project's PI, or an Admin giving Members ownership of their own contributions while keeping PIs and Admins in overall control.

### 6.4 Document Management

Documents are uploaded as real files (multipart/form-data), stored on the server, and served back via a static file endpoint. Upload is available to Members, PIs, and Admins; deletion is restricted to the owning project's PI or an Admin.

### 6.5 Admin Panel

Admins have access to a dedicated panel listing every registered user, with the ability to remove accounts. This is the only part of the application restricted to a single role end-to-end, on both the API and the UI.

## 6. Security Implementation

### 6.1 Security Design

- Spring Security is configured for stateless sessions for the API does not rely on server-side HTTP sessions.
- `/api/auth/` and uploaded-file paths are public in the current security configuration, other endpoints require a valid JWT.
- The JWT authentication filter runs before `UsernamePasswordAuthenticationFilter`.
- Fine-grained role checks use `@PreAuthorize` on controller methods and ownership checks in services where needed.
- CORS permits the local React origin (`http://localhost:3000`) and standard CRUD HTTP methods.
- The frontend stores the JWT in `localStorage`, attaches it as a Bearer token through an Axios request interceptor, and removes it/redirects to `/login` after a 401 response.
- Frontend role-aware UI hides or disables actions that the logged-in role cannot perform.

### 6.2 Authentication Mechanism

- Passwords are hashed with BCrypt before storage for plaintext passwords are never persisted.
- JWTs are signed with HMAC-SHA256 using a 256-bit secret, and expire after 24 hours.
- A custom `JwtAuthenticationFilter` runs once per request, ahead of Spring Security's standard filter chain.
- Unauthenticated or invalid-token requests receive a structured 401 JSON response rather than a default HTML error page.

### 6.3 Role-Based Access Control

Authorization is enforced with `@PreAuthorize` annotations at the controller layer, backed by ownership checks in the service layer where the rules depend on data, not just role. The permission matrix implemented is,

| Action                | ADMIN | PI      | MEMBER | VIEWER |
|-----------------------|-------|---------|--------|--------|
| View projects         | ✓     | ✓       | ✓      | ✓      |
| Create / edit project | ✓     | ✓ (own) | ✗      | ✗      |

| Action               | ADMIN | PI | MEMBER | VIEWER |
|----------------------|-------|----|--------|--------|
| Delete project       | ✓     | ✗  | ✗      | ✗      |
| Add / edit milestone | ✓     | ✓  | ✓      | ✗      |
| Upload document      | ✓     | ✓  | ✓      | ✗      |
| Manage users         | ✓     | ✗  | ✗      | ✗      |

## 6.4 Error Handling

A centralised RestControllerAdvice converts exceptions into consistent JSON error responses across the whole API → 404 for missing resources, 403 for access-denied, 401 for authentication failures, 400 with field-level detail for validation errors, and 500 as a safety net.

## 7. Backend API Reference (REST API Design)

### 7.1 Spring Boot Project Setup and Configuration

The backend is located in the `research-tracker-backend` directory. Its package root is `lk.ijse.cmjd.researchtracker`, divided into `auth`, `user`, `project`, `milestone`, `document`, `config`, and `common/exception` packages. Maven defines the Spring Boot dependencies and the application starts through `ResearchTrackerApplication`.

The current application configuration uses port 8081, connects to a local MySQL database named `research_tracker`, enables Hibernate schema updates, configures JWT token validity, sets the upload directory, and limits multipart file size/request size to 10 MB. Development values should be replaced with environment-specific settings for non-coursework deployments.

### 7.2 Authentication and JWT Flow

| Method | Endpoint                      | Description                              |
|--------|-------------------------------|------------------------------------------|
| POST   | <code>/api/auth/signup</code> | Register a new user (defaults to MEMBER) |
| POST   | <code>/api/auth/login</code>  | Authenticate and receive a JWT           |

1. A user submits username/password to POST `/api/auth/login`, or registration data to POST `/api/auth/signup`.
2. The authentication service validates credentials; registration creates a MEMBER role by default.
3. Passwords are stored as BCrypt hashes rather than plaintext.
4. On success, the backend generates a signed JWT containing identity/role information and returns an authentication response.
5. The client stores the JWT and sends it in the `Authorization: Bearer <token>` header on protected requests.
6. The backend JWT filter validates the token for each protected request and establishes the authenticated principal.
7. Method-level `@PreAuthorize` rules and service ownership checks determine whether the requested action is allowed.

### 7.3 Project Management

`ProjectController` exposes list, details, create, update, status update, and delete endpoints. Creating and updating projects is limited to PI and ADMIN roles at the controller level, project deletion is

ADMIN-only. The service layer additionally ensures that a PI can update only the project they own, while an ADMIN can manage any project. Project status values are PLANNING, ACTIVE, ON\_HOLD, COMPLETED, and ARCHIVED.

| Method | Endpoint                  | Description                              |
|--------|---------------------------|------------------------------------------|
| GET    | /api/projects             | List all projects                        |
| GET    | /api/projects/{id}        | View a single project                    |
| POST   | /api/projects             | Create a project (PI, Admin)             |
| PUT    | /api/projects/{id}        | Update a project (owning PI, Admin)      |
| PATCH  | /api/projects/{id}/status | Change project status (owning PI, Admin) |
| DELETE | /api/projects/{id}        | Delete a project (Admin only)            |

## 7.4 Milestone Management

Milestones are accessed under a project for listing/creation and through milestone-specific endpoints for updates/deletion. The frontend supports creating a milestone, editing its title/description/due date, toggling completion, and deleting it. Permission checks allow the administrator, the owning PI, or the milestone creator to modify a milestone according to the implementation rules.

| Method | Endpoint                      | Description                             |
|--------|-------------------------------|-----------------------------------------|
| GET    | /api/projects/{id}/milestones | List milestones for a project           |
| POST   | /api/projects/{id}/milestones | Add a milestone (Member, PI, Admin)     |
| PUT    | /api/milestones/{id}          | Update a milestone (creator, PI, Admin) |
| DELETE | /api/milestones/{id}          | Delete a milestone (creator, PI, Admin) |

## 7.5 Document Management

Documents are uploaded as multipart/form-data with a title, description, and file. DocumentService works with FileStorageService to save the file and persist its metadata. The React project details page constructs FormData, posts it to `/projects/{id}/documents`, lists uploaded documents, and opens the stored URL/path in a new browser tab. Current backend configuration caps uploaded files at 10 MB.

| Method | Endpoint                     | Description                           |
|--------|------------------------------|---------------------------------------|
| GET    | /api/projects/{id}/documents | List documents for a project          |
| POST   | /api/projects/{id}/documents | Upload a document (Member, PI, Admin) |
| DELETE | /api/documents/{id}          | Delete a document (PI, Admin)         |

## 7.6 User Administration

UserController provides user listing, profile lookup, and deletion. The frontend `/admin`` route is restricted to ADMIN users. The backend remains authoritative through ADMIN-only security on user listing/deletion operations.

| Method | Endpoint        | Description                 |
|--------|-----------------|-----------------------------|
| GET    | /api/users      | List all users (Admin only) |
| GET    | /api/users/{id} | View a user profile         |
| DELETE | /api/users/{id} | Delete a user (Admin only)  |

## 7.7 Exception Handling and Supporting Components

The backend includes a centralized common/exception package and a JWT authentication entry point so clients receive structured errors instead of application stack traces. Supporting configuration also includes CORS for the frontend origin and a DataSeeder that creates development accounts for ADMIN, PI, and VIEWER roles so role-protected features can be demonstrated during coursework testing.

## 8. Frontend Application Structure

### 8.1 Routing and Navigation

| Route              | Page / Purpose                                             | Access                            |
|--------------------|------------------------------------------------------------|-----------------------------------|
| /login             | Sign in                                                    | Public                            |
| /register          | Sign up (new accounts default to MEMBER)                   | Public                            |
| /dashboard         | Overview — project status summary and recent activity      | Authenticated                     |
| /projects          | Project list, with search and status filtering             | Authenticated                     |
| /projects/new      | Create project (PI, Admin)                                 | PI / ADMIN                        |
| /projects/:id      | Project detail — milestones and documents for that project | Authenticated; actions role-gated |
| /projects/:id/edit | Edit project (owning PI, Admin)                            | Owning PI / ADMIN                 |
| /milestones        | Milestones across all projects                             | Authenticated                     |
| /documents         | Documents across all projects                              | Authenticated                     |
| /admin             | User management (Admin only)                               | ADMIN only                        |

### 8.2 Key Design Decisions

- The top-level router places authenticated pages behind a `PrivateRoute` and places the admin page behind a `RoleRoute` that permits only the ADMIN role. This mirrors the backend authorization model and improves the user experience by preventing navigation to inaccessible pages.
- Global auth state is managed through a single `AuthContext`, backed by the JWT stored in `localStorage`; the token is decoded client-side to drive role-based UI without extra API calls.
- An `Axios` interceptor attaches the JWT to every outgoing request and redirects to `/login` automatically on a 401 response.
- Route guards (`PrivateRoute`, `RoleRoute`) enforce authentication and role restrictions at the routing level, in addition to the buttons/actions themselves being conditionally rendered by role.
- A consistent visual identity (navy and gold palette, paired serif/sans typefaces, status-colour-coded project cards) is applied across every page via a shared stylesheet.

## 8.3 Frontend Implementation

### 8.3.1 React Project Setup and Route Composition

The frontend is located in **research-tracker-frontend**. The ``src`` directory contains API helpers, reusable components, context providers, typed models, utility functions, and route-oriented page components. `App.tsx` defines public login/register routes, protected authenticated routes, and an ADMIN-only route for user management.

### 8.3.2 Authentication Context and Token Lifecycle

`AuthContext` owns the client authentication state. On startup it checks `localStorage` for a token and rehydrates the user only when the token is present and not expired. Successful login or registration stores the returned token and user information. Logout removes the token and clears the authenticated user state.

A shared `Axios` instance reads the token before each request and sets the `Authorization` header. Its response interceptor watches for HTTP 401, when encountered, it removes the token and redirects the browser to ``/login``. This satisfies the frontend assignment requirement to attach JWTs to backend requests and redirect when the token becomes invalid or expired.

### 8.3.3 Login and Registration

The login page uses React Bootstrap form validation, username/password state, an error alert, and a loading spinner while the request is in progress. After successful authentication the user is redirected either to the originally requested protected route or to ``/dashboard``. The login implementation corresponds to the supplied development URL below.

**Login implementation target** → <http://localhost:3000/login>

### 8.3.4 Project Pages and Dashboard

The frontend contains `DashboardPage`, `ProjectsListPage`, `ProjectFormPage`, and `ProjectDetailPage`. Authenticated users can browse project information, while project creation/editing controls are shown only when the role and ownership rules permit. Project list/search/filter behavior is separated from the project form and details screen, supporting reusable route-focused components.

### 8.3.5 Project Details, Status, Milestones, and Documents

ProjectDetailPage loads a project together with its milestones and documents. It calculates role/ownership flags such as whether the current user is an ADMIN, the owning PI, or a role allowed to create milestones/documents. The page displays the project status using a status badge, permits authorized users to update status or edit the project, and exposes ADMIN-only project deletion.

The same page contains milestone and document sections. Milestones can be added/edited/completed/deleted under the implemented permission rules. Documents can be uploaded through multipart FormData and listed as links to the backend-served file. The supplied project URL is the primary target for later screenshot-based testing evidence.

**Provided project details target** → <http://localhost:3000/projects/279d8037-2a4d-4569-b890-60958a914c55>

### 8.3.6 Role-Based UI and Administration

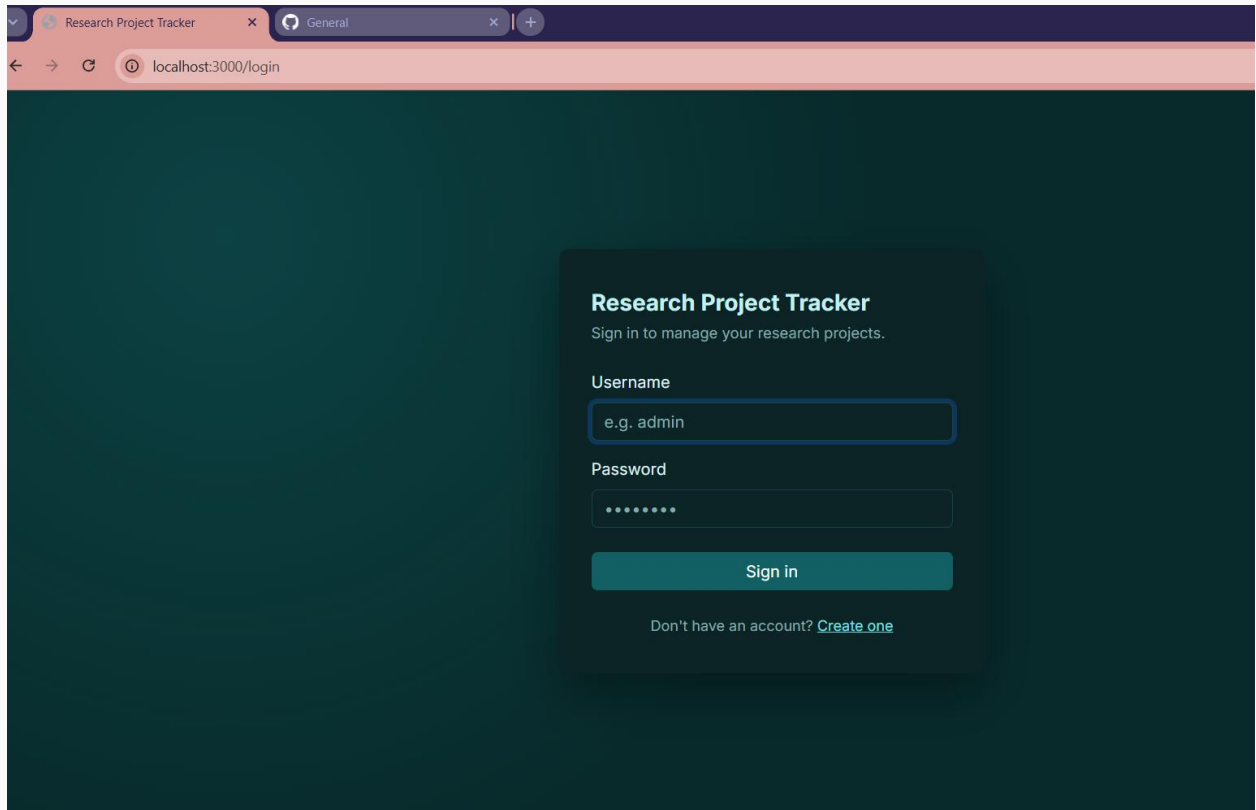
The frontend uses both route-level and component-level role awareness. `/admin` is wrapped in an ADMIN-only role route. Project action buttons, status selectors, milestone actions, and document actions are rendered conditionally based on the current user and project ownership. This reduces user confusion and complements the backend security checks.

### 8.3.7 Responsive Design, Loading, and Error Feedback

React Bootstrap and the shared CSS/layout components provide responsive form and page structures. Reusable loading indicators are displayed while data is fetched, while API errors are converted into user-readable messages. These behaviors meet the frontend brief requirements for responsive desktop/mobile layouts, spinners/loaders, and form validation.

## 9. Application Screenshots

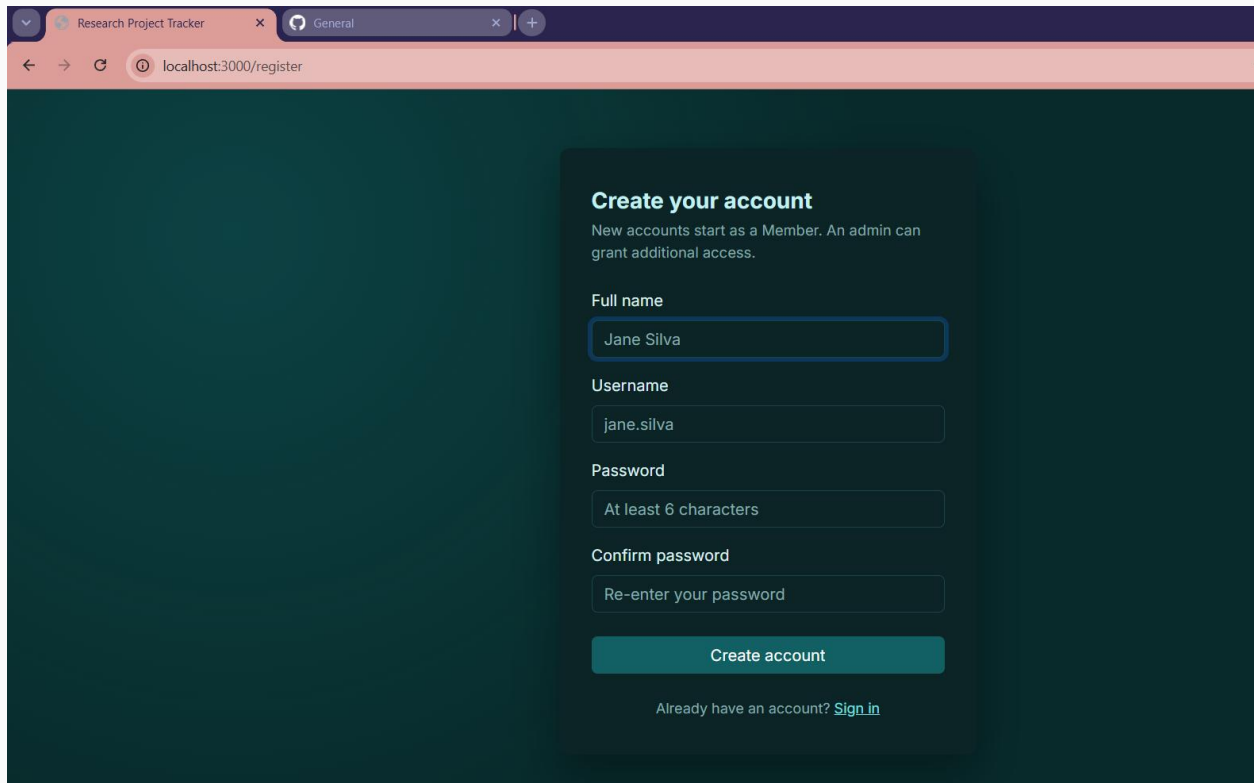
### 9.1 Sign In



**Login Page**

*<http://localhost:3000/login>*

## 9.2 Sign Up

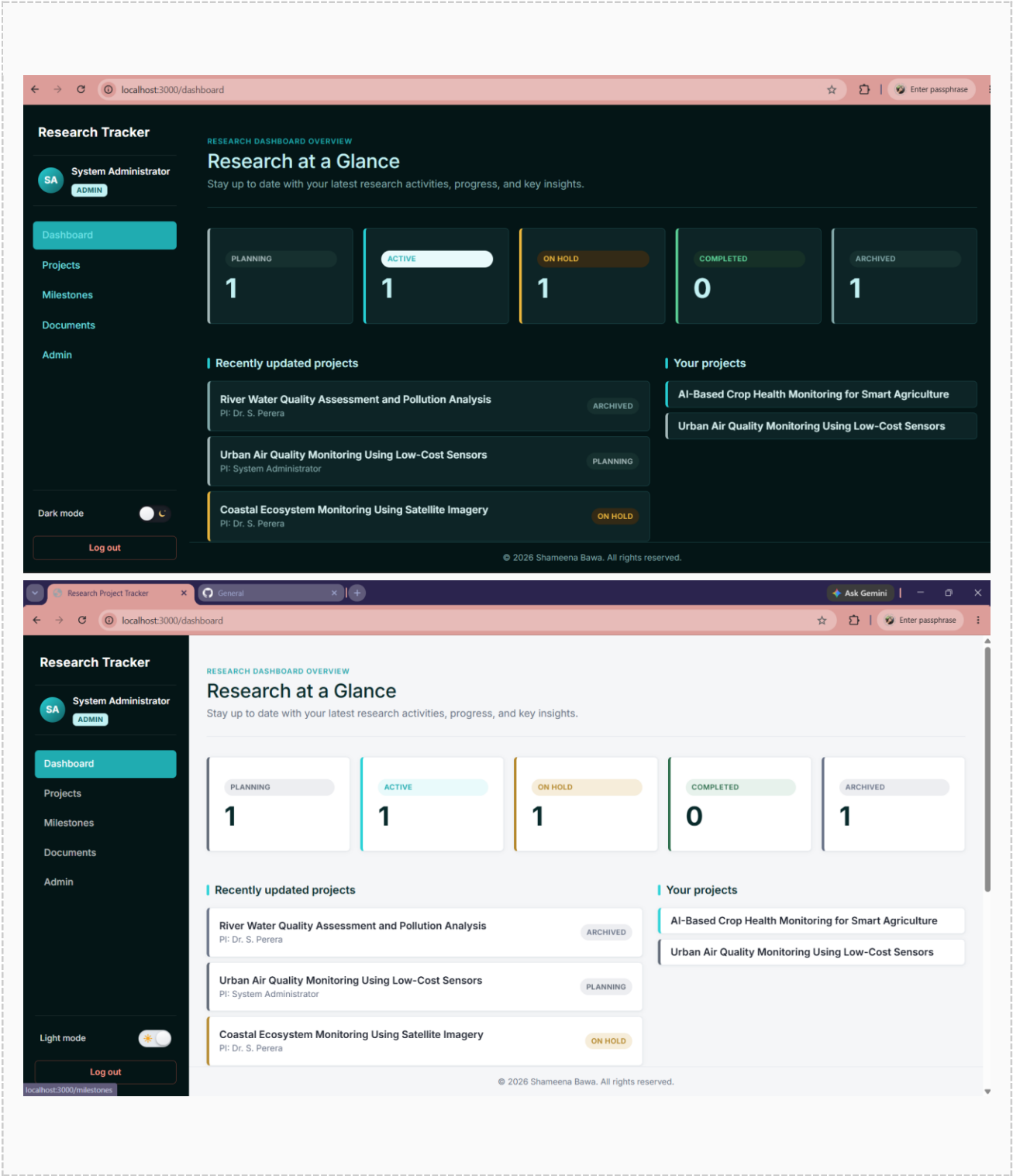


The screenshot shows a web browser window with two tabs: 'Research Project Tracker' and 'General'. The address bar displays 'localhost:3000/register'. The main content area has a dark teal background. Centered on this background is a dark gray registration form titled 'Create your account'. Below the title, a subtitle reads: 'New accounts start as a Member. An admin can grant additional access.' The form contains four input fields: 'Full name' (with 'Jane Silva' entered), 'Username' (with 'jane.silva' entered), 'Password' (with the placeholder 'At least 6 characters'), and 'Confirm password' (with the placeholder 'Re-enter your password'). A teal 'Create account' button is positioned below the inputs. At the bottom of the form, a link says 'Already have an account? [Sign in](#)'.

### Register Page

*<http://localhost:3000/register>*

9.3 Dashboard



The screenshot displays the Research Tracker dashboard. The browser address bar shows `localhost:3000/dashboard`. The dashboard has a dark theme and a sidebar on the left with navigation links: Dashboard (selected), Projects, Milestones, Documents, and Admin. The user is logged in as System Administrator (SA) with an ADMIN role. The main content area is titled "RESEARCH DASHBOARD OVERVIEW" and "Research at a Glance". It features a card for "AI-Based Crop Health Monitoring for Smart Agriculture" with a status of "ACTIVE". Below this, there are two sections: "Milestones" and "Documents". The "Milestones" section shows 12 pending and 12 total milestones, with a "View all" button. The "Documents" section shows 12 uploaded documents, also with a "View all" button. The dashboard includes a "Log out" button and a "Dark mode" toggle. The footer shows the URL `localhost:3000/projects/279d8037-2a4d-4569-b890-60958a914c55` and a copyright notice: "© 2026 Shameena Bawa. All rights reserved."

Research Tracker

System Administrator  
SA ADMIN

Dashboard

Projects

Milestones

Documents

Admin

Dark mode

Log out

RESEARCH DASHBOARD OVERVIEW

## Research at a Glance

Stay up to date with your latest research activities, progress, and key insights.

**AI-Based Crop Health Monitoring for Smart Agriculture**  
PI: System Administrator ACTIVE

### Milestones

12 pending · 12 total View all

|                                                                                                            |                |
|------------------------------------------------------------------------------------------------------------|----------------|
| <b>Sampling Locations and Baseline Study</b><br>River Water Quality Assessment and Pollution Analysis      | Due 2026-09-30 |
| <b>Satellite Data Collection</b><br>Coastal Ecosystem Monitoring Using Satellite Imagery                   | Due 2026-10-15 |
| <b>Air Quality Data Collection</b><br>Urban Air Quality Monitoring Using Low-Cost Sensors                  | Due 2026-12-31 |
| <b>Water Quality Data Collection and Analysis</b><br>River Water Quality Assessment and Pollution Analysis | Due 2026-12-31 |
| <b>Coastal Change Analysis</b><br>Coastal Ecosystem Monitoring Using Satellite Imagery                     | Due 2027-01-15 |

### Documents

12 uploaded View all

|                                                                                                     |           |
|-----------------------------------------------------------------------------------------------------|-----------|
| <b>AI Model Evaluation Report</b><br>AI-Based Crop Health Monitoring for Smart Agriculture          | 8/14/2026 |
| <b>Crop Image Dataset Documentation</b><br>AI-Based Crop Health Monitoring for Smart Agriculture    | 8/14/2026 |
| <b>Smart Agriculture Research Proposal</b><br>AI-Based Crop Health Monitoring for Smart Agriculture | 8/14/2026 |
| <b>Coastal Change Analysis Report</b><br>Coastal Ecosystem Monitoring Using Satellite Imagery       | 8/14/2026 |
| <b>Satellite Imagery Dataset Report</b><br>Coastal Ecosystem Monitoring Using Satellite Imagery     | 8/14/2026 |

localhost:3000/projects/279d8037-2a4d-4569-b890-60958a914c55

© 2026 Shameena Bawa. All rights reserved.

## Dashboard Overview

*<http://localhost:3000/dashboard>*

## 9.4 Projects List

The image displays two screenshots of the Research Tracker application. The top screenshot shows the 'Projects' list page, and the bottom screenshot shows the 'Start a new research project' form.

**Top Screenshot: Projects List**

The browser address bar shows `localhost:3000/projects`. The application header includes a search bar and a '+ New project' button. The left sidebar shows the user is logged in as 'System Administrator' (SA) with an 'ADMIN' role. The sidebar menu includes 'Dashboard', 'Projects' (selected), 'Milestones', 'Documents', and 'Admin'. The main content area is titled 'RESEARCH PROJECTS Projects' and includes a search bar 'Search by title, tag, or PI...' and a dropdown for 'All statuses'. The projects are listed in a grid:

- AI-Based Crop Health Monitoring for Smart Agriculture** (ACTIVE)  
This project explores the use of artificial intelligence and sensor data to monitor crop health. The research will identify early signs of p...  
PI: System Administrator | AI - agriculture - machine learning
- Urban Air Quality Monitoring Using Low-Cost Sensors** (PLANNING)  
This project aims to monitor and analyze air quality in urban areas using low-cost environmental sensors. The study will collect data on par...  
PI: System Administrator | AI - air quality - environmental science
- River Water Quality Assessment and Pollution Analysis** (ARCHIVED)  
This project investigates the quality of freshwater resources by analyzing physical and chemical water-quality indicators. The study will id...  
PI: Dr. S. Perera | Water quality - pollution - environmental science
- Coastal Ecosystem Monitoring Using Satellite Imagery** (ON HOLD)  
This project aims to analyze coastal ecosystems using satellite imagery and environmental data. The study will monitor shoreline changes, ve...  
PI: Dr. S. Perera | Satellite imagery - coastal science - environment

The bottom of the page shows a copyright notice: © 2026 Shameena Bawa. All rights reserved.

**Bottom Screenshot: Start a new research project**

The browser address bar shows `localhost:3000/projects/new`. The left sidebar is identical to the top screenshot. The main content area is titled 'NEW PROJECT Start a new research project' and contains a form with the following fields:

- Title**: e.g. Coastal Erosion Monitoring with Satellite Imagery
- Summary**: A short description of the project's goals and scope
- Tags**: AI, environment, coastal science (Comma-separated)
- Start date**: dd/mm/yyyy
- Expected end date**: dd/mm/yyyy
- Principal Investigator**: Assign myself as PI (dropdown menu)

At the bottom of the form are 'Create project' and 'Cancel' buttons. The footer shows the same copyright notice: © 2026 Shameena Bawa. All rights reserved.

The screenshot shows a web browser at `localhost:3000/projects/new` displaying the 'Research Tracker' interface. The left sidebar contains a user profile for 'System Administrator' (SA) with an 'ADMIN' role, and a menu with 'Dashboard', 'Projects' (highlighted), 'Milestones', 'Documents', and 'Admin'. At the bottom of the sidebar are 'Dark mode' and 'Log out' buttons. The main content area is titled 'NEW PROJECT' and 'Start a new research project'. It contains several input fields: 'Title' (with placeholder 'e.g. Coastal Erosion Monitoring with Satellite Imagery'), 'Summary' (with placeholder 'A short description of the project's goals and scope'), 'Tags' (with placeholder 'AI, environment, coastal science' and a note 'Comma-separated.'), 'Start date' and 'Expected end date' (both with 'dd/mm/yyyy' format and calendar icons), and 'Principal Investigator' (with a dropdown menu showing 'Assign myself as PI' and a note 'As an admin, you may assign this project to any PI.'). At the bottom of the form are 'Create project' and 'Cancel' buttons. A footer at the very bottom reads '© 2026 Shameena Bawa. All rights reserved.'

## Projects List Page

*<http://localhost:3000/projects>*

## 9.5 Project Detail

The screenshot shows the 'Research Tracker' application interface. The main heading is 'Milestones' with the subtitle 'ACROSS ALL PROJECTS'. Below this, it states 'Every milestone tracked across the institute's research projects.' There is a filter button 'Pending only' and an '+ Add milestone' button. The table below lists various milestones, each with a status icon, a milestone name, a project name, a due date, and the user who added it.

| Status | Milestone                                  | Project                                                               | Due date   | Added by             |
|--------|--------------------------------------------|-----------------------------------------------------------------------|------------|----------------------|
| 🕒      | Sampling Locations and Baseline Study      | <a href="#">River Water Quality Assessment and Pollution Analysis</a> | 2026-09-30 | System Administrator |
| 🕒      | Satellite Data Collection                  | <a href="#">Coastal Ecosystem Monitoring Using Satellite Imagery</a>  | 2026-10-15 | System Administrator |
| 🕒      | Air Quality Data Collection                | <a href="#">Urban Air Quality Monitoring Using Low-Cost Sensors</a>   | 2026-12-31 | System Administrator |
| 🕒      | Water Quality Data Collection and Analysis | <a href="#">River Water Quality Assessment and Pollution Analysis</a> | 2026-12-31 | System Administrator |
| 🕒      | Coastal Change Analysis                    | <a href="#">Coastal Ecosystem Monitoring Using Satellite Imagery</a>  | 2027-01-15 | System Administrator |
| 🕒      | AI Model Development                       | <a href="#">AI-Based Crop Health Monitoring for Smart Agriculture</a> | 2027-01-31 | System Administrator |
| 🕒      | Pollution Assessment and Final Report      | <a href="#">River Water Quality Assessment and Pollution Analysis</a> | 2027-02-05 | System Administrator |
| 🕒      | Data Analysis and Final Report             | <a href="#">Urban Air Quality Monitoring Using Low-Cost Sensors</a>   | 2027-02-28 | System Administrator |
| 🕒      | Results and Environmental Assessment       | <a href="#">Coastal Ecosystem Monitoring Using Satellite Imagery</a>  | 2027-03-15 | System Administrator |
| 🕒      | Model Testing and Evaluation               | <a href="#">AI-Based Crop Health Monitoring for Smart Agriculture</a> | 2027-04-30 | System Administrator |
| 🕒      | Sensor Setup and Study Area Selection      | <a href="#">Urban Air Quality Monitoring Using Low-Cost Sensors</a>   | 2027-09-30 | System Administrator |
| 🕒      | Data Collection and Preparation            | <a href="#">AI-Based Crop Health Monitoring for Smart Agriculture</a> | 2027-10-31 | System Administrator |

The sidebar on the left contains the following navigation items: Dashboard, Projects, Milestones (highlighted), Documents, and Admin. The top header shows the user 'SA System Administrator' with a 'Log out' button. The footer indicates '© 2026 Shameena Bawa. All rights reserved.'

Add milestone

Project

AI-Based Crop Health Monitoring for Smart Agriculture

Title

Description

Due date

dd/mm/yyyy

Cancel

Add milestone

**Research Tracker**

System Administrator (SA) ADMIN

Dashboard  
Projects  
Milestones  
Documents  
Admin

Light mode

Log out

ACROSS ALL PROJECTS

## Milestones

Every milestone tracked across the institute's research projects.

Pending only + Add milestone

**Add milestone**

Project: [AI-Based Crop Health Monitoring for Smart Agriculture](#)

Title:

Description:

Due date:

Cancel Add milestone

| Status | Milestone                                                       | Due date   | Added by             |
|--------|-----------------------------------------------------------------|------------|----------------------|
|        | <a href="#">Sampling Locations and Baseline Data Collection</a> | 2026-09-30 | System Administrator |
|        | <a href="#">Satellite Data Collection</a>                       | 2026-10-15 | System Administrator |
|        | <a href="#">Air Quality Data Collection</a>                     | 2026-12-31 | System Administrator |
|        | <a href="#">Water Quality Data Collection</a>                   | 2026-12-31 | System Administrator |
|        | <a href="#">Coastal Change Analysis</a>                         | 2027-01-15 | System Administrator |
|        | <a href="#">AI Model Development</a>                            | 2027-01-31 | System Administrator |
|        | <a href="#">Pollution Assessment and Final Report</a>           | 2027-02-05 | System Administrator |
|        | <a href="#">Data Analysis and Final Report</a>                  | 2027-02-28 | System Administrator |
|        | <a href="#">Results and Environmental Assessment</a>            | 2027-03-15 | System Administrator |
|        | <a href="#">Model Testing and Evaluation</a>                    | 2027-04-30 | System Administrator |
|        | <a href="#">Sensor Setup and Study Area</a>                     | 2027-09-30 | System Administrator |
|        | <a href="#">Data Collection and Preparation</a>                 | 2027-10-31 | System Administrator |

© 2026 Shameena Bawa. All rights reserved.

**Research Tracker**

System Administrator (SA) ADMIN

Dashboard  
Projects  
Milestones  
Documents  
Admin

Dark mode

Log out

ACROSS ALL PROJECTS

## Documents

Reference materials and files uploaded to research projects.

+ Upload document

| Title                                 | Project                                                               | Uploaded by          | Uploaded at                        |
|---------------------------------------|-----------------------------------------------------------------------|----------------------|------------------------------------|
| AI Model Evaluation Report            | <a href="#">AI-Based Crop Health Monitoring for Smart Agriculture</a> | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Crop Image Dataset Documentation      | <a href="#">AI-Based Crop Health Monitoring for Smart Agriculture</a> | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Smart Agriculture Research Proposal   | <a href="#">AI-Based Crop Health Monitoring for Smart Agriculture</a> | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Coastal Change Analysis Report        | <a href="#">Coastal Ecosystem Monitoring Using Satellite Imagery</a>  | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Satellite Imagery Dataset Report      | <a href="#">Coastal Ecosystem Monitoring Using Satellite Imagery</a>  | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Coastal Research Proposal             | <a href="#">Coastal Ecosystem Monitoring Using Satellite Imagery</a>  | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Air Quality Data Collection Report    | <a href="#">Urban Air Quality Monitoring Using Low-Cost Sensors</a>   | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Sensor Specifications and Setup Guide | <a href="#">Urban Air Quality Monitoring Using Low-Cost Sensors</a>   | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Research Proposal                     | <a href="#">Urban Air Quality Monitoring Using Low-Cost Sensors</a>   | System Administrator | 8/14/2026 <a href="#">Download</a> |
| River Pollution Assessment Report     | <a href="#">River Water Quality Assessment and Pollution Analysis</a> | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Water Sampling and Laboratory Report  | <a href="#">River Water Quality Assessment and Pollution Analysis</a> | System Administrator | 8/14/2026 <a href="#">Download</a> |
| Water Quality Research Proposal       | <a href="#">River Water Quality Assessment and Pollution Analysis</a> | System Administrator | 8/14/2026 <a href="#">Download</a> |

© 2026 Shameena Bawa. All rights reserved.

Upload document

Project

AI-Based Crop Health Monitoring for Smart Agriculture

Title

Description

File

Choose File

No file chosen

Cancel

Upload

Project Detail — Milestones and Documents

<http://localhost:3000/projects/279d8037-2a4d-4569-b890-60958a914c55>

## 9.6 Admin Panel

The image displays two screenshots of the Research Project Tracker Admin Panel, showing the user management interface in both light and dark themes.

**Light Mode Screenshot:**

- Header:** Research Tracker, System Administrator (SA), ADMIN.
- Left Sidebar:** Dashboard, Projects, Milestones, Documents, Admin (selected), Light mode toggle, Log out.
- Main Content:** ADMIN PANEL, User management, Full list of registered accounts across the system. + Create user button.
- User List Table:**

| Full name            | Username         | Role   | Joined    |        |
|----------------------|------------------|--------|-----------|--------|
| Dr. S. Perera        | @dr.perera       | PI     | 8/13/2026 | Delete |
| Nethmi Jayawardena   | @nethmi.j        | VIEWER | 8/14/2026 | Delete |
| Guest Viewer         | @viewer1         | VIEWER | 8/13/2026 | Delete |
| System Administrator | @admin           | ADMIN  | 8/13/2026 | Delete |
| Sarah Wijesinghe     | @sarah.w         | MEMBER | 8/14/2026 | Delete |
| Amanda Perera        | @amanda.perera   | ADMIN  | 8/14/2026 | Delete |
| Daniel Fernando      | @daniel.fernando | PI     | 8/14/2026 | Delete |

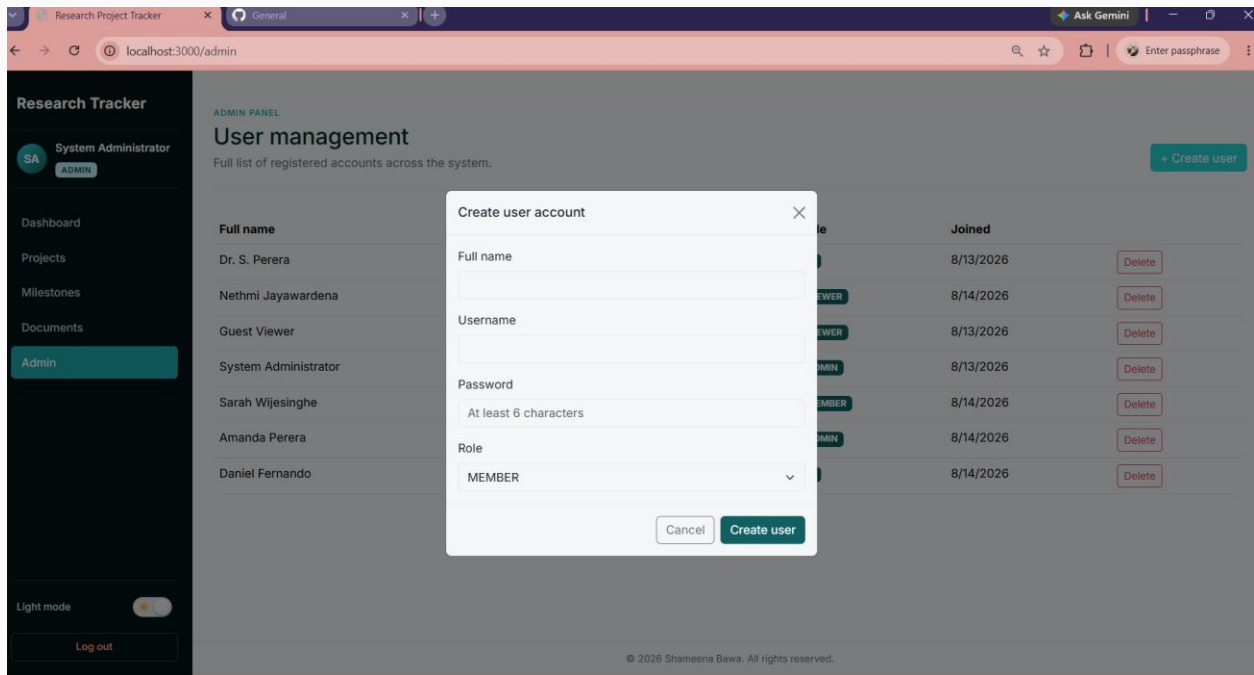
© 2026 Shameena Bawa. All rights reserved.

**Dark Mode Screenshot:**

- Header:** Research Tracker, System Administrator (SA), ADMIN.
- Left Sidebar:** Dashboard, Projects, Milestones, Documents, Admin (selected), Dark mode toggle, Log out.
- Main Content:** ADMIN PANEL, User management, Full list of registered accounts across the system. + Create user button.
- User List Table:**

| Full name            | Username         | Role   | Joined    |        |
|----------------------|------------------|--------|-----------|--------|
| Dr. S. Perera        | @dr.perera       | PI     | 8/13/2026 | Delete |
| Nethmi Jayawardena   | @nethmi.j        | VIEWER | 8/14/2026 | Delete |
| Guest Viewer         | @viewer1         | VIEWER | 8/13/2026 | Delete |
| System Administrator | @admin           | ADMIN  | 8/13/2026 | Delete |
| Sarah Wijesinghe     | @sarah.w         | MEMBER | 8/14/2026 | Delete |
| Amanda Perera        | @amanda.perera   | ADMIN  | 8/14/2026 | Delete |
| Daniel Fernando      | @daniel.fernando | PI     | 8/14/2026 | Delete |

© 2026 Shameena Bawa. All rights reserved.



### User Management (Admin only)

<http://localhost:3000/admin>

## 10. Setup and Installation

---

### 10.1 Prerequisites

- JDK 17 or higher, Maven
- Node.js 18 or higher, npm
- MySQL 8, running locally

### 10.2 Backend

Create a MySQL database, update the credentials in `application.properties`, then run,

```
mvn spring-boot:run
```

The API starts on `http://localhost:8080`. Database tables are created automatically on first run.

### 10.3 Frontend

```
npm install && npm start
```

The application opens on `http://localhost:3000` and connects to the backend automatically.

## 11. System Integration, Deployment Environment and Version Control

### 11.1 End-to-End Integration Flow

| Step | Action                   | System Interaction                                                                      |
|------|--------------------------|-----------------------------------------------------------------------------------------|
| 1    | User opens /login        | React renders the LoginPage.                                                            |
| 2    | User submits credentials | AuthContext calls Axios POST /api/auth/login.                                           |
| 3    | Backend authenticates    | Spring Security/authentication service validates credentials and returns JWT.           |
| 4    | Client stores JWT        | AuthContext stores token and user; browser navigates to dashboard.                      |
| 5    | User opens project       | React GETs /api/projects/{id} and related milestone/document endpoints with Bearer JWT. |
| 6    | Backend authorizes       | JWT filter establishes user; controller/service checks role/ownership.                  |
| 7    | Data is persisted/read   | JPA repositories interact with MySQL; file service handles uploads.                     |
| 8    | UI updates               | Axios responses update React state and display project/milestone/document data.         |

### 11.2 Version Control and Run Environment

The complete project source code, including both the Spring Boot backend and React frontend, is maintained using Git and hosted on GitHub. The repository also contains the necessary setup and execution documentation.

The main environment requirements are

- Java 17+
- Maven
- Node.js 18+
- npm
- MySQL 8

For local execution, the backend runs on port 8081, while the React frontend runs on port 3000. The project also includes seeded user accounts for demonstrating the authentication and role-based authorization features.

**GitHub Repository:** <https://github.com/shami95/ResearchProjectTracker>

## 12. Conclusion

---

The Research Project Tracker successfully implements a full-stack web application using Spring Boot, MySQL, React, and TypeScript. The system provides secure JWT-based authentication, BCrypt password protection, role-based authorization, and CRUD operations for managing projects, milestones, and documents.

The application follows a clear separation between the frontend, backend, business logic, and database layers. The React frontend provides a responsive and role-aware user interface, while the Spring Boot backend manages authentication, authorization, data processing, and persistence. Different access levels for ADMIN, PI, MEMBER, and VIEWER roles ensure that users can only perform actions permitted by their responsibilities.

Overall, the project demonstrates the successful integration of modern full-stack technologies, secure application development practices, database management, and role-based access control. It meets the main coursework objectives and provides a solid foundation for future enhancements and extensions.

## 13. Recommendations

---

- 1 Externalize database credentials and JWT signing secrets before any non-local deployment.
- 2 Add automated backend tests (e.g., Spring Boot/JUnit/MockMvc) and frontend tests for auth, route guards, and critical forms.
- 3 Add refresh-token or more secure session handling if the project evolves beyond coursework; localStorage is simple but has trade-offs for production security.
- 4 Add file-type allow-listing, malware scanning, and stricter storage controls for production document uploads.
- 5 Use database migrations (Flyway/Liquibase) instead of relying only on Hibernate ddl-auto=update for controlled deployments.
- 6 Add pagination/search at API level if project/user counts grow substantially.
- 7 Add audit logging for important administrative and project status changes.
- 8 Update the repository run guide so all backend-port examples consistently reflect the configured port.